Guess Close, Search Exact:
A Formally Verified, Model-Free Memory and Mutation Engine for Language-Model Agents

Devieswar Kancheti
Independent researcher

Code, data and every run: `github.com/devkancheti4-design/neo4d`

October 2026

Abstract

Language models are good at proposing roughly right answers and poor at being exactly right, cheaply, every time. We present *neo4d*, a model-free engine that sits beside a model and supplies what the model lacks: it mutates code token by token and lets the tests judge every variant, it searches the neighbourhood of a model's attempt, and it remembers verified work as placeholder plans it can replay without the model. What it trusts is decided by *RELATE v2*, an eight-bit integer law with no branches, checked on all 2^{32} inputs for undefined behaviour (UBSan and CBMC); its nine principles hold on every reachable input, and eight further properties on all 256 eight-bit inputs, including that a pattern marked barren is weakened but never erased and that a proven pattern is never demoted. Under a teacher that gives 10% wrong lessons, the law keeps 58.4% of what memory produces correct, against 0.9% when every pattern is trusted at first sight and 1.7% without its barren rule, at equal coverage. We measure the engine with local 4B models, with Claude Haiku and Claude Sonnet run as agents through the same tools, and against a Claude assistant. With neo4d, phi4-mini fixes 62.5% of 176 bugs (30.7% alone, 42.0% with three tries) and completes 47.1% of 85 multi-step agent tasks (25.9% alone, 40.0% with three tries) with fewer calls than retrying. Sonnet and Haiku keep their accuracy (85/85, 84/85) while neo4d takes 32% and 40% of the tasks off them; a replayed plan takes 0.59 s against 20–36 s for a model. With Haiku only choosing which branches to explore, neo4d's search fixes 17 hard two-fault bugs where a blind search fixes 14 and a random filter 12.0 on average, with 60% fewer test runs. On real bugs at an equal budget over three seeds, a loop in which neo4d mutates each of the model's attempts fixes 24.7 of 31 with gemma3:4b against 16.0–18.7 for sample-and-filter, self-debugging and Reflexion ($p \leq 0.016$) with about 35% fewer calls; with phi4-mini it leads (23.0 against 20.0–22.3) without a significant difference. On two domains that are not code—a numbers puzzle and block-stacking plans—the same core, unchanged, lifts both small models far above their own methods (Countdown: from 0 to 26–31 of 40), but its search alone, with no model, does as well or better: there the small models' guesses are worse starts than a neutral one. We report equally plainly what failed: evidence handed to a small model as a paragraph made it worse, marks on suspect lines helped one model and hurt another, and nothing helped on problems a strong model already solves. neo4d adds power by doing work and remembering it, not by advising.

1 Introduction

An agent built on a language model spends most of its budget on two things the model does badly: being exactly right (a one-token slip fails the tests) and doing the same kind of work again (a rename across files costs as many calls the tenth time as the first). Both are things a model-free system can do well, if it has something to tell right from wrong. Tests are that something.

neo4d is built on this division of labour. The model *guesses*: it reads, proposes a direction, writes. neo4d *checks and remembers*: it expands a guess into every single-token variant its mutation operators know, runs the tests on each, and keeps verified work as plans with the task's names abstracted into slots. An integer law, judged exhaustively and verified formally, decides which plans and patterns are trusted, which are only primed, and which have proved barren.

Contributions.

1. **A verified memory law.** RELATE v2 maps a level and six bits of evidence to one of four trust levels with 48 branch-free instructions. We give its principles, the exhaustive and formal checks, the properties that govern barren patterns, and what is *not* proved (§3). Under a noisy teacher it keeps 58.4% precision where trusting at first sight keeps 0.9% and the same law without its barren rule 1.7%.
2. **Guess close, search exact.** A model's attempt need only land within one token of a fix: neo4d's neighbourhood search and the tests do the rest. At an equal budget over three seeds it beats sample-and-filter, self-debugging and Reflexion significantly with gemma3:4b and leads without significance with phi4-mini, with fewer calls for both (§5.3).
3. **Plans that bypass the model.** Verified tool traces become placeholder macros, matched by the task's phrasing with names removed, replayed and verified in 0.59 s. They double a small model on agent work and take a third to two fifths of the work off Haiku and Sonnet (§5.2).
4. **A model as a filter.** Inverting the roles—neo4d generates and tests, a model only picks branches—improves a search and cuts its cost (§5.4).
5. **A domain-free core.** The loop, the search, memory and the law in one file that names no domain; two small adapters carry it to a numbers puzzle and to planning (§5.6).
6. **Negative results**, reported with the same care (§6), and every run released.

2 The engine

Mutation. Each code line is tokenised and changed one token at a time by the standard operators of mutation testing [1, 2]: relational ($< \leftrightarrow \leq \leftrightarrow = \ \ldots$), arithmetic, logical connectors and negation, constants ($0 \leftrightarrow 1$, `True`↔`False`), control flow (`break`↔`continue`, `return x`→`return None`), and, in a harder set, argument, index, deletion and unary insertion. On the programs used here this gives about 3.4 variants per code line.

Grading. Every variant runs against every test case in a forked process with a time limit. For each case neo4d records pass or fail and the value returned (or the exception's type). Three counts follow: *fixed* (failing cases that now pass), *broken* (passing cases that now fail) and *moved* (failing cases whose output changed at all). *Moved* localises a fault, as in mutation-based fault localisation [3]; *fixed*−*broken* ranks near-fixes. A program's own printed output is discarded; there is no partial credit inside a case.

Plans. From accepted work—a sequence of tool actions (`grep`, `replace`, `write`, `test`, …)—neo4d writes the task's names into slots. A task names things in backticks; its *skeleton* replaces each with $\langle i \rangle$. Names become slots at identifier boundaries in code, between non-alphanumerics in paths (`tests/test_`$\langle 1 \rangle$`.py`), and around regular-expression escapes in patterns (`\b`$\langle 2 \rangle$`\(`). A step repeated over every file a search found becomes one looping step; an answer equal to the number, paths or module names of the files found becomes a derived value. A step whose text a plan cannot fill (the code a fix writes) is marked for the model to write. Matching is exact equality of skeletons; every replay runs on a fresh copy of the project and is accepted or discarded whole.

The exchange. Given a model's attempt, neo4d mutates it—the lines the model changed first—and ships the first variant that passes every test; otherwise it reports what the attempt and its best variants pass, and the model goes again (§5.3).

Deep search. neo4d expands a program by every single-token change, keeps $B = 3$ candidates, and expands those again, up to four changes deep, with duplicates skipped. Kept candidates are chosen either by tests passed (*blind*) or by a model shown the 12 most promising, at most two per line (*filter*). The cost is about $M + dBM$ test runs for M single-token variants and depth d—linear in depth, not M^d.

3 RELATE v2: a verified law for what to trust

Each pattern—a relation, a plan—carries a level: 0 unknown, 1 primed, 2 active (used), 3 proven. At every lesson the body computes six bits of evidence and the law returns the new level:

$$x \;=\; L \mid \textsc{fire}{\ll}2 \mid \textsc{newnet}{\ll}3 \mid \textsc{pred}{\ll}4 \mid \textsc{cheap}{\ll}5 \mid \textsc{costly}{\ll}6 \mid \textsc{barren}{\ll}7.$$

FIRE: the lesson exhibits the pattern; NEWNET: in a place (bug, file, module) it was not seen in; PRED: the pattern, used, was right; CHEAP/COSTLY: what using it costs (for a plan, model calls; in the mutation domain, candidates per line and how many are wrong); BARREN: it has fired at least k times ($k = 10$ here, 100 for bugs) with under 5% success. The law was authored outside this system from a written specification of nine principles (R1 a costly pattern is never used; R2 silence keeps; R3 first sight primes; R4 a cheap pattern is used at first sight unless R9; R5 a second place generalises; R6 a correct prediction potentiates; R7 proven only by prediction; R8 nothing else raises; R9 a barren pattern weakens unless proven). It compiles to 48 instructions with no conditional branches and no memory operations.

What is verified.

- All 4,294,967,296 `int32` inputs under UBSan: no undefined behaviour, every output in 0..3, and the output depends only on the low eight bits. CBMC [6] proves the same, with no signed overflow.
- The nine principles hold on all 114 reachable words, with 0 violations; no single-cell change to the table escapes them.
- On all 256 eight-bit inputs, the properties of Table 1 hold.

property (all 256 inputs)	holds
A barren, unpredicted, unproven pattern is never active (level ≤ 1)	✓
Barrenness weakens but never erases: never below primed, at most one step down	✓
A proven pattern is never demoted by barrenness (only COSTLY caps it)	✓
A new place cannot re-activate a barren pattern; only a correct prediction can	✓
Without BARREN or COSTLY a level never decreases (monotone)	✓
A correct prediction always makes a pattern active, unless costly	✓
A costly pattern is never active	✓
With the evidence held fixed, every trajectory reaches a fixed point (no oscillation)	✓

Table 1: Properties of RELATE v2, checked by enumeration of its whole input space.

What is not proved. The BARREN bit is a statistic computed by the body; the law guarantees what happens once a pattern is marked (it is weakened, not erased, and can return only by predicting correctly), not that the marking is right. Convergence of the whole system depends on the stream of evidence, not on the law alone.

Why it matters: an ablation under a noisy teacher. With a teacher that gives wrong lessons, the barren rule is what keeps memory clean. On 10,000 lessons of the mutation domain (mutants of the Python standard library), evaluated on held-out modules over three seeds, we compare RELATE v2 with the same law ignoring its BARREN bit and with no law (every pattern used at first sight, never weakened). Coverage is the share of real mutants memory can produce; precision the share of what it produces that is real (Table 2). The earlier version of the law, which let a cheap pattern survive barrenness, behaves like the ablated one (1.9% at 10%, 0.9% at 50%).

wrong lessons	rule	coverage	**precision** (range over seeds)	patterns kept
0%	RELATE v2	96.3%	**82.0%** (71.5–87.9)	438
	v2 without BARREN	96.4%	81.6% (71.1–87.4)	440
	no law	96.4%	81.6% (71.1–87.4)	440
10%	RELATE v2	96.3%	**58.4%** (48.9–73.5)	467
	v2 without BARREN	96.3%	1.7% (1.4–2.2)	830
	no law	96.3%	0.9% (0.7–1.2)	858
30%	RELATE v2	95.9%	**27.3%** (19.8–32.6)	514
	v2 without BARREN	96.1%	0.9% (0.8–1.2)	1,274
	no law	96.1%	0.6% (0.5–0.7)	1,313

Table 2: A noisy teacher. Without the barren rule, wrong patterns accumulate and almost everything memory produces is wrong; with it, precision holds at equal coverage and memory stays half the size.

4 Experimental setup

Models. phi4-mini and gemma3:4b run locally through ollama on one Apple-silicon machine (an M4 Pro, 24 GB), temperature 0 for a first try and 0.8 for later ones (qwen3.5:4b was started and stopped as too slow; no result of it is reported). Claude Haiku and Claude Sonnet run as agents (subagents) through exactly the tools the local models use, one action per command. A Claude assistant (Opus-class, the system used to build and run this study) answered a sample under a handicap (two rounds of actions instead of up to ten).

Benchmarks. All built on the 31 QuixBugs [5] programs whose tests are JSON cases.

- **Bug stream** (176 tasks): each correct program broken on one line by the mutation operators, kept only if its own tests catch it, in one seeded order.
- **Agent work** (85 tasks): the programs as a project (`qb/`, one test file each under `tests/`); six kinds of task—rename a function across files, rename a parameter, add a keyword parameter, count modules using a keyword in code (docstrings excluded), list the modules calling a built-in, fix a bug—each phrased two ways, each accepted only by the tests plus a check of the result itself. Tools: `ls`, `read`, `grep`, `replace`, `write`, `test`, `done`; at most ten actions.
- **Real bugs** (31): the human-written QuixBugs bugs. neo4d's mutations alone fix 5; 26 remain.
- **Hard bugs** (29): each correct program obfuscated (hash-named variables, no docstrings), its function renamed to an opaque name, and broken by two faults on two lines, each fault alone failing the tests, no single change fixing the pair. 18 are reachable by neo4d's operators; 11 contain a fault outside them.
- **Not code** (40 tasks per seed, seeds 1 and 2): *Countdown*—reach a target with four given numbers and $+ - \times \div$, checked by exact evaluation—and *Blocks world*—restack 3–5 named blocks into a goal, checked by a simulator. Both are generated; the checker says right or wrong and how close, never the answer.

5 Results

5.1 Bug stream: memory that compounds

phi4-mini, 176 tasks	fixed	original line exactly	model calls
model alone	30.7%	29.0%	176
model + tests (3 tries)	42.0%	38.1%	407
neo4d + model	**62.5%**	58.5%	**251**

Table 3: neo4d's memory proposes fixes first; a single passing proposal ships with no model call.

Fixes from memory rose from 38.6% of the first half of the stream to 45.5% of the second; 71 of 74 restored the original line exactly. A perfect writer scores 166/176 through the same harness (its line-placement heuristic loses `return x`→`return None` bugs). On the first 30 tasks the assistant fixed 28—the harness ceiling—with

one answer each, against 7 for phi4-mini alone and 15 with neo4d; neo4d + the assistant also fixed 28, with 19 calls instead of 30.

5.2 Agent work: plans that bypass the model

85 tasks	accepted	second half	model calls	tasks needing the model
phi4-mini alone	25.9%	30.2%	579	85
phi4-mini + 3 tries	40.0%	44.2%	1,406	85
neo4d + phi4-mini	**47.1%**	**55.8%**	**1,225**	–
Claude Sonnet alone	85/85	43/43	418	85
neo4d + Sonnet	**85/85**	43/43	**285**	**58**
Claude Haiku alone	84/85	43/43	401	85
neo4d + Haiku	**84/85**	43/43	**226**	**51**

Table 4: For the large models, model calls are their actions; "neo4d + model" replays trusted plans that need no model and uses the model's own recorded work for every other task (a model starts each task fresh, so its work does not depend on neo4d). Plans of equal strength are tried in a fixed order, so these numbers reproduce exactly.

neo4d's plans completed 3 tasks alone in the first half of the stream and 13 in the second. In the second half neo4d took 19 of 43 tasks off Sonnet and 22 of 43 off Haiku. A replayed plan takes a median 0.59 s, of which 0.46 s is the full test run that verifies it; phi4-mini needs about 20 s per task and the subagents 27–36 s (wall clock, including tool overhead)—35 to 60 times slower. On a 28-task sample the assistant completed 28 (two rounds, one try), phi4-mini 9 alone and 16 with neo4d; neo4d + the assistant completed 28 with the assistant needed on 19.

By kind, neo4d's plans did most for renaming a parameter (9 tasks with no model call) and nothing for renaming a function: phi4-mini completed one rename in fifteen, and the plan it left did not carry to the others.

Ablation. Replaying the Sonnet and Haiku work with plans whose names are kept literal instead of slotted, neo4d takes no task off either model (85 of 85 need the model, against 58 and 51). Replacing RELATE v2 by no law, or ignoring its barren rule, changes nothing here: plans that need no model call are cheap and trusted at first sight under every rule, and no plan fires the ten times a barren mark needs. In this setting the slots do the work; the law matters under noise (Table 2).

5.3 Round after round until it answers

On the 31 real bugs, two loops of up to ten rounds that differ only in neo4d: *test feedback* (the model is told what its last program passes and the first failing case) and *model* $\rightleftarrows$ *neo4d* (the same, and neo4d mutates the model's program, ships any passing variant, and reports its best variants). Neither is given evidence about the original bug up front. A first single run favoured neo4d (Table 5).

one run, up to 10 rounds	test feedback	calls	model $\rightleftarrows$ neo4d	calls
phi4-mini	22	154	25	120
gemma3:4b	18	151	24	98

Table 5: A single run on the 31 real bugs (fixed of 31).

Against equal-budget baselines, with seeds. We then gave every method at most ten model calls per bug and ran three seeds: *sample-and-filter* (independent tries, the first that passes [11]), *self-debugging* [9] (the test-feedback loop), *Reflexion* [10] (after a failure the model writes a short reflection, kept for later attempts; a reflection costs a call) and the neo4d loop (Table 6). The interval is a 95% bootstrap over bugs

of the per-bug success rate averaged over seeds; p is a paired sign-flip permutation test of neo4d against the method.

model	method	fixed (seeds 0, 1, 2)	mean	95% interval	calls per bug	neo4d − method (p)
phi4-mini	sample-and-filter	23, 22, 22	22.3	58–85%	4.8	+0.7 (0.84)
	self-debugging	20, 22, 22	21.3	55–83%	5.0	+1.7 (0.52)
	Reflexion	19, 23, 18	20.0	49–78%	5.3	+3.0 (0.24)
	neo4d loop	20, 25, 24	**23.0**	60–87%	**4.0**	–
gemma3:4b	sample-and-filter	16, 16, 16	16.0	34–69%	5.5	+8.7 (0.004)
	self-debugging	19, 18, 19	18.7	43–76%	4.7	+6.0 (0.016)
	Reflexion	18, 19, 19	18.7	44–76%	4.9	+6.0 (0.008)
	neo4d loop	25, 24, 25	**24.7**	65–92%	**3.2**	–

Table 6: Every method at most ten model calls per bug, 31 real bugs. With gemma3:4b the neo4d loop fixes six to nine more bugs than every baseline, significantly, with about a third fewer calls; with phi4-mini it leads without a significant difference, with a fifth fewer calls.

Over three seeds and against the strongest baselines, the single-run gain in Table 5 holds for gemma3:4b and shrinks to fewer calls for phi4-mini.

5.4 A model as a filter for neo4d's search

29 hard bugs	fixed	reachable (18)	unreachable (11)	model calls	test runs
blind (keeps the 3 passing most tests)	14	13	1	0	5,815
random filter (5 seeds; range 10–14)	12.0	11.4	–	0	6,484
Haiku as the filter	**17**	**16**	1	37	**2,316**
Haiku writing the code, for reference	29	18	11	29	–

Table 7: neo4d generates and tests every program; Haiku only picks which branches go deeper.

The filter fixed every bug the blind search fixed and three more, and more than every random seed: its picks carry information that neither the test counts nor chance do. Writing code directly, Haiku fixed all 29: these are small classical algorithms it recognises even obfuscated; the filter's ceiling is neo4d's operators.

5.5 Clarity for a small model's attention

The first design gave phi4-mini neo4d's evidence as a paragraph before the bug. We also show the same evidence as a single comment on the one or two lines neo4d's mutations point to (Table 8). Marks lifted phi4-mini's first try from 11 to 15 but not its three tries (18 against 20), and hurt gemma3:4b (13 against 15 at one try); the paragraph hurt both. Pointing a small model's attention is not a reliable gain.

shown	one try	up to 3 tries	calls
phi4-mini, plain	11	20	97
phi4-mini, evidence as a paragraph	12	16	98
phi4-mini, marks on suspect lines	**15**	18	93
gemma3:4b, plain	**15**	**15**	94
gemma3:4b, evidence as a paragraph	11	12	102
gemma3:4b, marks on suspect lines	13	14	97

Table 8: The same bug shown plainly, with evidence as a paragraph, and with marks on the suspect lines (fixed of 31).

5.6 The same core on domains that are not code

The loop, the search, memory and the RELATE v2 law live in one file that names no domain (a test checks it); a domain supplies only how to ask, read, check, vary and abstract. Two design runs (recorded in the repository) changed the puzzle sizes, the prompts (the same for every method), the checker's partial score and the search depth; the configuration was then frozen and run on seeds 1 and 2 (Table 9). Every method had at most four model calls per task; neo4d's methods also had a budget of checker calls.

domain, model	checks	model only (best)	**neo4d core**	search, no model	core's calls
Countdown, phi4-mini	100	0.0	**8.5**	29.0	3.6
	400	0.0	**31.0**	33.5	2.1
Countdown, gemma3:4b	100	0.0	**6.5**	29.0	3.5
	400	0.0	**26.0**	33.5	2.0
Blocks, phi4-mini	100	4.0	3.0	29.5	3.7
	400	4.0	7.5	31.5	3.5
Blocks, gemma3:4b	100	10.0	12.0	29.5	2.9
	400	10.0	**23.5**	31.5	2.0

Table 9: Solved of 40 per seed, mean over two seeds. The core lifts the models far above their own methods (significantly in the bold rows, $p < 0.001$), but search alone, with no model, does as well or better in every row.

Neither 4B model solved a single Countdown puzzle on its own, with one try, with four, or with feedback; with the core they solved up to 31 of 40. But a neutral start beats their guesses: their arithmetic is wrong and their plans break the rules, so search spent near them is search wasted. Memory solved some tasks with no model call (up to 12.5 of 40 for gemma3:4b in Blocks), but removing it, or the law, changed little: the checkers are exact, so no lesson was wrong. The core carries across domains unchanged; whether it raises a model there depends on the model landing close, which these did not.

6 What did not work

- **Advice as a paragraph.** With neo4d's evidence before the bug, phi4-mini fixed 16 of 31 with three tries, against 20 without it; an exchange of three rounds with that evidence reached 18. A small model's attention is diluted by text it must weigh.
- **Marks on suspect lines.** Helped phi4-mini's first try, hurt gemma3:4b and phi4-mini's retries.
- **More fixes from the loop, for phi4-mini.** Over three seeds its lead over sample-and-filter (23.0 against 22.3) is not significant; its single-run win over self-debugging shrank to calls.
- **A small model as the proposer in puzzles.** On Countdown and Blocks world the 4B models' guesses were worse starting points than a neutral one: neo4d's search alone beat the core with a model in every setting.
- **The law on repeated, clean work.** On the agent tasks no rule beat any other; the law's value appears only when lessons can be wrong.
- **Helping a model that already knows.** Haiku fixed all 31 real bugs at the first submission with or without neo4d, and all 29 hard bugs—at the first submission without neo4d, while with neo4d's evidence two needed a second.
- **Vocabulary for search.** An exploratory test of code search (descriptions to functions over click, rich and pytest) found that learned word-to-code links changed first-time search accuracy by -0.4 to -0.2 points; links to remembered functions helped searching again ($+4.5$ points) and hurt first searches. It was stopped.
- **Nothing from nothing.** A plan needs a success that generalises: phi4-mini renamed a function across files once in fifteen tasks, and the plan did not carry to the others. Eleven of the hard bugs lie outside neo4d's operators and no search of them can fix them.

7 Threats to validity

The benchmarks are small (31, 85, 176 tasks). Most configurations were run once; the noisy teacher, the random control, the equal-budget baselines and the core on other domains have seeds, and the baselines show that a single-run difference of three bugs can shrink. The non-code domains are generated puzzles, chosen and adjusted in two recorded design runs before the reported seeds. QuixBugs is public, and every model may have seen it; the hard set reduces but does not remove recall. Two benchmarks inject bugs with the same operators neo4d uses to repair, which favours neo4d's memory and search; the real-bug and agent results do not have this circularity in the bug itself. The agent tasks recur by design, which is where plans pay. The phi4-mini agent run was made before plan ties were ordered deterministically. The assistant that built the study also answered a sample and belongs to the same family as two evaluated models. A check (the tests and a check of the result) is assumed; without one neo4d cannot tell right from wrong.

8 Related work

Mutation testing [1, 2] supplies the operators and mutation-based fault localisation [3] the *moved* signal. Search-based repair such as GenProg [4] evolves patches against tests; neo4d instead searches the neighbourhood of a model's attempt and remembers what passed. Learning-based repair has moved to large models [7, 8], and self-debugging [9] and Reflexion [10] feed test results or reflections back to the model; our test-feedback loop is that baseline. Sampling many programs and filtering by tests [11] spends model calls on breadth; neo4d spends none. Skill libraries [12] and agent-computer interfaces [13] store or shape what agents do; neo4d's plans are verified, slot-abstracted and governed by a formally checked law. The memory law is checked with CBMC [6].

9 Conclusion

A model need not be exactly right if something beside it can search exactly and remember. Under a law whose behaviour is proved over its whole input space and which keeps memory clean when a teacher is wrong, neo4d doubles a small model on agent work, takes a third or more of repeated work off large models at equal accuracy, beats strong equal-budget baselines on real bugs with one small model (and matches them with another) using fewer calls, carries unchanged to domains that are not code, and makes a model useful as a cheap filter for a search it could not run itself. It does not make a model reason better by advising it, and it cannot create a skill no one has shown it. Next: a stronger proposer on the non-code domains, coverage-guided search inside the agent, and SWE-bench, whose agent and guide are released with the code.

Reproducibility. Every number comes from a command in the repository: `neo4d local-model`, `local-agent` (with `--agent-score sonnet,haiku`), `local-hints` (with `--loop`, `--focus`, `--search-score`), `python -m neo4d.rigor` (`--law`, `--agent`, `--random`, `--baselines`), `python -m neo4d.core_experiment`. Every action of the Claude subagents and every local run is in `results/data/`.

Use of AI. The implementation, experiments and text were produced with Claude Code (Anthropic) under the author's direction; the author chose the questions and the claims. Claude models are also subjects of evaluation here.

References

[1] R. A. DeMillo, R. J. Lipton, F. G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer* 11(4), 1978.

[2] Y. Jia, M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37(5), 2011.

[3] M. Papadakis, Y. Le Traon. Metallaxis-FL: mutation-based fault localization. *Software Testing, Verification and Reliability* 25(5–7), 2015.

[4] C. Le Goues, T. Nguyen, S. Forrest, W. Weimer. GenProg: a generic method for automatic software repair. *IEEE Transactions on Software Engineering* 38(1), 2012.

[5] D. Lin, J. Koppel, A. Chen, A. Solar-Lezama. QuixBugs: a multi-lingual program repair benchmark set based on the Quixey Challenge. *SPLASH Companion*, 2017.

[6] E. Clarke, D. Kroening, F. Lerda. A tool for checking ANSI-C programs. *TACAS*, 2004.

[7] C. S. Xia, Y. Wei, L. Zhang. Automated program repair in the era of large pre-trained language models. *ICSE*, 2023.

[8] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, S. H. Tan. Automated repair of programs from large language models. *ICSE*, 2023.

[9] X. Chen, M. Lin, N. Schärli, D. Zhou. Teaching large language models to self-debug. *ICLR*, 2024.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: language agents with verbal reinforcement learning. *NeurIPS*, 2023.

[11] Y. Li et al. Competition-level code generation with AlphaCode. *Science* 378, 2022.

[12] G. Wang et al. Voyager: an open-ended embodied agent with large language models. arXiv:2305.16291, 2023.

[13] J. Yang et al. SWE-agent: agent-computer interfaces enable automated software engineering. *NeurIPS*, 2024.